

ECE 157B/272B - Homework 1

ECE 157/272 TAs

Due Date: Friday, January 24th, 2025, 11:59PM

Start early!

Introduction

In the last quarter, we explored traditional machine learning algorithms and image-based machine learning algorithms. We practiced with each algorithm by building models using numerical data sets with Python's Scikit-Learn library and using image data sets with the PyTorch library. In this quarter, we will continue our machine learning journey and explore the path of text-based machine learning algorithms and various language models. In this first homework, we will introduce you to the basic building blocks of language models and the techniques to handle text-based dataset.

In Part A, we will be training simple autoregressive (GPT-style) language models (SimpleRNN, LSTM, and GRU) on the entirety of the works of Shakespeare. The data set is a collection of plays by William Shakespeare, formatted as a stage play script. We will train language models on this dataset and then generate text from these models to determine if and how well it has learned the structure of the plays, the structure of language, and the structure of the dataset.

In Part B, we will adapt the code from Part A to train a language model on the `daily_dialog` dataset from HuggingFace, which has a training set only slightly larger than the complete works of Shakespeare. We will compare the vocabularies generated by the two datasets, digging further into how tokenization works. Then we will train one vocabulary on both datasets and use that vocabulary to train two language models, one for each dataset, before comparing the final perplexities of the two models on each of their datasets. (Note: You can only compare perplexities between models trained in the same tokenizer and vocabulary![2] Even some experienced researchers, such as the peer reviewer rebutted in [1], have made this mistake.)

Finally, we will train one more language model on the `daily_dialog` dataset, but this time we'll train it on the classification task of categorizing the emotion of each line of the conversation based on the emotion data provided in the original dataset, as an example of sentiment classification.

Dataset

Shakespeare plays

This comma-separated-values document has been preprocessed for you into a script-like raw text format which your language model will learn to approximately replicate. Each line is either a line delimited with three apostrophes indicating the start of a new play or the name of the speaking character, followed by a colon and the character's line. You can view this script file with any text editor. A snippet of a play is shown below:

From `shakespeare.txt`

''' PLAY: A Comedy of Errors '''

NARRATOR: ACT I

NARRATOR: SCENE I. A hall in DUKE SOLINUS'S palace.

NARRATOR: Enter DUKE SOLINUS, AEGEON, Gaoler, Officers, and other Attendants

AEGEON: Proceed, Solinus, to procure my fall

AEGEON: And by the doom of death end woes and all.

daily_dialog

The zipped dataset has been extracted for you into a file of scripts and a separate file of emotion vectors, pre-split into training and validation sets. Each script is written on a single line with each line of dialog followed by the demarcator “__eou__”. The emotions are listed in corresponding lines in the emotion file with the emotion for each segment of the script separated by a space. A few short corresponding lines of the two files are shown below:

From `daily_dialog_train_text.txt`

```
I'm so nervous about this oral test . __eou__ Just calm down . I'm sure you'll do well . __eou__ I just want to get it over with . __eou__  
I had a wonderful evening . __eou__ I did , too.Come and visit us again , will you ? __eou__ I will . __eou__  
I'm terribly nervous . __eou__ Calm down ! __eou__
```

From `daily_dialog_train_emotion.txt`

```
3 0 0  
4 0 0  
3 0
```

Note that the emotion data comes as an enumeration of 6 values:

0. No emotion (and sometimes just no label)
1. Anger
2. Disgust
3. Fear
4. Happiness
5. Sadness
6. Surprise

Part A Tasks

We will be building SimpleRNN, LSTM, and GRU language models with minimal complexity to perform language modeling for the complete works of Shakespeare. The code implementation of the whole workflow is given in the `Homework1A.ipynb` notebook. Read the comments in the code to understand the workflow, as this will help you with tweaking the models we’ve provided. Then answer the following questions in your lab report:

1. Experiment with at least one value of the `VOCAB_SIZE` parameter other than the default 20,000. How does changing this value affect how much text is returned by the example batch decoding under *Generate input and target sequence*?
2. Our example code uses the WordPiece tokenizer to break up the text into numbered tokens for the model to learn. If you print the tokenizer vocabulary with `tokenizer.get_vocab()` you will find that some tokens start with a “##”. What does this represent to WordPiece during text encoding and decoding?
3. What does the `forward` method of our recurrent model object do? Implement this function. (You need only go into the level of detail as shown in the provided code – you do not have to look up the functionalities of the underlying PyTorch sub-modules. Do describe in brief what purpose each is accomplishing.)
4. When training, PyTorch makes a “graph” of all the operations we perform so that it can automatically compute the derivatives of the different neural net weights with respect to the loss. Why do we need to `detach` this graph from our state variables between batches? Why not start over with a blank state?

5. Create a model using the `create_model` function *without* training it with `train_model`. Use this model to generate some text. Does this text appear structured? Does it at least seem Shakespearean? Include at least two samples of at least 20 tokens each and comment on the output.
6. Train a model using `model_type=RNN`, `model_type=LSTM`, `model_type=GRU`. For each model, record one generated text of at least 10 tokens that you found interesting for understanding the model behavior. How does each model's output compare to each other? What happens if you set `by_prob=False`? `by_prob=True`?

For each model training experiment, preserve the training code you used in your notebook for your own later reference. This helps to prevent repeating the same training experiment multiple times.

Part B Tasks

We will be experimenting on vocabularies from the two datasets, then training models on each dataset with a shared vocabulary. Finally, we'll use the emotion data from `daily_dialog` to train a dialog emotion classifier. The code implementation of the whole workflow is given in the `Homework1B.ipynb` notebook. Read the comments in the code to understand the workflow, as this will help you with tweaking the models we've provided. Then answer the following questions in your lab report:

1. Make up 3 sentences. Use the `tokenizer_toy` (as provided in the notebook) to encode and inspect the sentences with both a tokenizer trained on each of
 - (a) `data/shakespeare_train.txt` and
 - (b) `data/daily_dialog_train_text.txt`

and place the results in your report slides. Comment on which tokenizer handles each of your sentences "better". Specify what your definition of "better" is.

2. Use the provided notebook code to train three vocabularies:
 - `data/shakespeare_train.txt` only
 - `data/daily_dialog_train_text.txt` only
 - `data/shakespeare_train.txt` and `data/daily_dialog_train_text.txt` both together

Consider the differences in token length and number of tokens between the three vocabularies. Then answer explicitly: why are the peak count of token lengths (that is, the length of token that has the most distinct tokens *of* that length) different between the three vocabularies?

3. Use the provided notebook code to tokenize each dataset with each vocabulary. Examine the dataset size (after tokenization) for each dataset and vocabulary pair. Which dataset+vocabulary pairing has the largest dataset size? Why? Which has the smallest? Why?
4. Complete the `forward` function in `RecurrentModel` from Part A to implement a recurrent language model. Note your choice of optimizer, model type, and parameters. Then train two language models, one on each dataset, tokenized using the tokenizer trained on both datasets. Test the perplexity of each model. Explain why the perplexities are different between the two models on the same dataset.
5. Complete the accuracy calculation code in `train_epoch_classifier` and `evaluate_classifier`. Note your choice of optimizer, model type, and parameters. Then train two language models, one pre-trained with the text generation task, and one randomly initialized, on the `daily_dialog` dataset for the emotion classification task. Test the accuracy of each model on the validation set. Explain why the accuracies are different between the two models on the same dataset.
6. Generate per-class accuracies and confusion matrices for your classifiers with the provided code. Consider why you might choose one classifier over the other.

What to turn in

- For the report, use the given slide template to make *slides* that show the results/submissions requested in the Tasks section. Export and submit the slides as a PDF document to GradeScope.
- Submit your code and classification results to the autograder on GradeScope.
 - NOTE: The grader is written to return feedback on your results as soon as it finishes running. Check your score to make sure the file is in the correct format and that you meet the threshold.

References

- [1] Anonymous. *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. Nov. 2023. URL: <https://openreview.net/forum?id=AL1fq05o7H¬eId=Ho2NSra71h>.
- [2] Sabrina J. Mielke. *Can you compare perplexity across different segmentations?* Apr. 2019. URL: <https://sjmielke.com/comparing-perplexities.htm>.